

An open Python implementation of the FRB/US macroeconomic model*

Vahid Ahmadi[†]

July 2026

Abstract

FRB/US is the Federal Reserve Board’s large-scale macroeconometric model of the United States, used by Board staff for forecasting and for monetary and fiscal policy analysis. The Board publishes the model’s full equation system, coefficients, quarterly database and a Python reference implementation (pyfrbus) in the public domain, but the reference package depends on pinned, ageing scientific-Python libraries. This paper documents an open, from-scratch, modern Python reimplementation of FRB/US with VAR (backward-looking) expectations, built directly from the Fed’s published `model.xml` and LONGBASE database as part of the PolicyEngine Macro programme. The implementation parses the 284-equation simultaneous system, constructs a sparse analytic Jacobian by symbolic differentiation, and solves each quarter with a damped Newton method using sparse LU factorisation. Validation is exact in the two senses that matter: after add-factoring, the solved baseline reproduces the LONGBASE trajectory for all 284 endogenous variables to machine precision (5.6×10^{-17}), and an identical 100-basis-point federal funds rate shock run in both this implementation and the Fed’s own pyfrbus agrees across all endogenous variables and quarters to a maximum absolute difference of 6.0×10^{-9} , with headline variables agreeing to 10^{-10} or better. The simulated monetary-policy and government-spending experiments reproduce the qualitative and quantitative properties of published FRB/US simulations: output falls about half a per cent roughly two years after a funds-rate shock, and the first-year spending multiplier is about 0.7 when the inertial Taylor rule is allowed to respond.

Keywords: FRB/US, macroeconometric models, monetary policy, fiscal policy, open-source software, model validation

***Acknowledgements:** I thank Max Ghenis for his great suggestions and helpful discussions. This paper and the accompanying software were developed with AI assistance (Anthropic’s Claude); all results were generated by running the released code and all errors are my own.

[†]Research Associate at PolicyEngine. Email: vahid@policyengine.org.

1 Introduction

FRB/US is the Federal Reserve Board’s large-scale quarterly macroeconometric model of the United States. Introduced in 1996 as the successor to the MPS model, it is one of the principal models that Board staff consult for forecasting and for the analysis of monetary and fiscal policy issues (Brayton and Tinsley, 1996; Brayton et al., 2014). Unusually among central-bank workhorse models, FRB/US is public: the Board publishes the complete equation system with estimated coefficients (`model.xml`), the quarterly historical-plus-projection database (`LONGBASE.TXT`), extensive technical documentation, and reference simulation platforms in EViews and, since 2021, in Python (the `pyfrbus` package), all in the public domain (Board of Governors of the Federal Reserve System, 2026b,a).

This paper documents `frbus`, an open, from-scratch reimplement of the FRB/US simulation platform in modern Python, developed as part of PolicyEngine Macro — PolicyEngine’s programme of open macroeconomic modelling infrastructure that complements its tax–benefit microsimulation models. The motivation is practical. The Fed’s own `pyfrbus` reference package is an invaluable specification of the model’s semantics, but it is pinned to an ageing scientific-Python stack (`sympy` 1.3, `scikit-umfpack`, pre-2.0 `pandas`) and does not install cleanly on current interpreters without patching. A dependable, continuously tested implementation on current `numpy/scipy/sympy/pandas` lowers the barrier to using FRB/US in research and in downstream policy tooling, where microsimulation-based revenue and distributional estimates can be combined with model-consistent macroeconomic feedbacks.

The reimplement is deliberately conservative: it treats the Fed’s published `model.xml` and `LONGBASE` data as the single source of truth and follows `pyfrbus` semantics exactly, while making independent numerical choices where these are equivalence-preserving (a single-block Newton solve of the full simultaneous system per period, rather than the vendor’s block decomposition). Fidelity is then demonstrated rather than asserted, in two layers. First, a *tracking invariant*: after computing add factors, solving the model over 2026Q1–2030Q4 reproduces the `LONGBASE` baseline for all 284 endogenous variables to 5.6×10^{-17} — machine precision. Second, *cross-validation*: the identical monetary-policy experiment run in this implementation and in the Fed’s `pyfrbus` (in a separate, patched environment) agrees across all 284 endogenous variables and all 20 quarters to a maximum absolute difference of 6.0×10^{-9} , with the headline variables — real GDP, prices, unemployment, the funds rate — agreeing to 10^{-10} or better. Both tests run as hard gates in continuous integration.

Beyond replication of the reference implementation, the model’s simulation properties line up with what the Fed has published about FRB/US itself. A 100-basis-point shock to the inertial Taylor rule lowers real GDP by 0.55 per cent at its trough seven quarters out, raises the unemployment rate by up to 0.26 percentage point, and lowers core PCE inflation by up to 0.034 percentage point — signs, magnitudes and timing consistent with the funds-rate-shock responses under VAR expectations reported by Brayton et al. (2014). A simple government-purchases experiment yields a first-year spending multiplier of about 0.7 with monetary policy responding through the inertial Taylor rule, in line with the model’s documented fiscal properties.

The remainder of the paper proceeds as follows. Section 2 summarises the history and structure of FRB/US, including the polynomial-adjustment-cost (PAC) framework and the expectations options, with representative equations. Section 3 describes the implementation. Section 4 presents the validation evidence. Section 5 presents simulation results and compares them with published FRB/US properties. Section 6 discusses limitations.

2 The FRB/US model

2.1 History and design

FRB/US was introduced at the Federal Reserve Board in 1996, replacing the MPS model that had been in use since the 1970s (Brayton and Tinsley, 1996). Its design responded to the Lucas-critique era: most behavioural equations are derived from explicit optimisation problems of households, firms and financial-market participants, with expectations appearing explicitly, while retaining the empirical fit and sectoral detail expected of a staff forecasting model. The current public model file specifies 367 variables, of which 284 are endogenous, split between behavioural equations and identities; the remainder are exogenous drivers, policy instruments and switches (Board of Governors of the Federal Reserve System, 2026a).

The model covers the standard expenditure and income sides of the U.S. economy: household consumption of non-durables and services (`eco`) and durables (`ecd`); residential (`eh`) and business fixed investment in equipment (`epd`, `epi`) and structures (`eps`); labour supply, hours (`lhp`) and unemployment (`lur`); a detailed price–wage block built around trend unit labour costs, with core PCE inflation (`picxfe`) linked to slack, trend inflation expectations and relative-price shocks; foreign trade; a fiscal sector with disaggregated federal and state–local taxes, transfers and purchases, plus debt- and surplus-targeting fiscal reaction functions; and a financial block mapping the federal funds rate into term and risk-adjusted rates, equity values and other asset prices.

2.2 Polynomial adjustment costs

The adjustment dynamics of most key non-financial variables are governed by the generalised adjustment-cost framework known as PAC — polynomial adjustment costs (Tinsley, 1993, 2002). An agent choosing y_t minimises

$$C_t = \sum_{i=0}^{\infty} \beta^i \left[(y_{t+i} - y_{t+i}^*)^2 + \sum_{k=1}^m b_k ((1-L)^k y_{t+i})^2 \right], \quad (1)$$

which penalises deviations of y from its desired level y^* and movements in the level and $m-1$ higher time-differences of y ; β ($= 0.98$) is a discount factor and L the lag operator. The Euler equation can be manipulated into the PAC decision rule

$$\Delta y_t = a_0 (y_{t-1}^{1*} - y_{t-1}) + \sum_{i=1}^{m-1} a_i \Delta y_{t-i} + \sum_{i=0}^{\infty} d_i \Delta y_{t+i}^{1*} + \sum_{i=0}^{\infty} h_i y_{t+i}^{0*}, \quad (2)$$

where the target has been decomposed into non-stationary (y^{1*}) and stationary (y^{0*}) components: the change in y responds to the lagged gap between y and the trend component of its target, to its own lags (reflecting higher-order frictions), and to discounted expected future movements in the target. In the model file the two expected-future sums are assigned to explicit expectations variables Z^1 and Z^0 ,

$$\Delta y_t = a_0 (y_{t-1}^{1*} - y_{t-1}) + \sum_{i=1}^{m-1} a_i \Delta y_{t-i} + Z_t^1 + Z_t^0, \quad (3)$$

augmented by a growth-neutrality correction so that $y = y^{1*}$ in a balanced-growth equilibrium. Most spending equations further combine optimising agents with rule-of-thumb agents whose

spending tracks income or output x :

$$\Delta y_t = \gamma \left(a_0 (y_{t-1}^{1*} - y_{t-1}) + \sum_{i=1}^{m-1} a_i \Delta y_{t-i} + Z_t^1 + Z_t^0 \right) + (1 - \gamma) \Delta x_t. \quad (4)$$

PAC is used in the equations for consumption (`eco`, `ecd`), fixed investment (`eh`, `epd`, `epi`, `eps`), aggregate hours (`lhp`) and dividends, and the same error-correction-with-expectations structure organises the price–wage block.

2.3 Expectations: VAR and MCE

FRB/US supports two treatments of the expectations variables Z^0 and Z^1 (Brayton et al., 2014). Under *model-consistent expectations* (MCE), the infinite forward sums are collapsed into finite-lead recursions in future Z ’s and solved jointly with the model’s own future paths. Under *VAR expectations* — the configuration implemented here — expectations are generated by small estimated vector autoregressions. Each PAC equation’s small model stacks the target components with a core three-variable VAR (output gap, inflation, funds rate) in companion form $z_t = H z_{t-1}$, and the expectations terms become linear functions of lagged observables:

$$Z_t^0 = h'_0 z_{t-1}, \quad Z_t^1 = h'_1 z_{t-1}, \quad (5)$$

where the coefficient vectors h_0 , h_1 are known nonlinear functions of the PAC coefficients, the discount factor and the VAR companion matrix H . Because these expectations depend only on current and lagged variables, the VAR version of the model is fully backward-looking: the 284-equation system is simultaneous *within* each quarter but recursive *across* quarters, which is what permits the period-by-period solution strategy of Section 3.

2.4 Policy rules

Fiscal policy is closed by reaction functions that adjust taxes to stabilise either the debt-to-GDP ratio (`dfpdbt`) or the fiscal surplus ratio (`dfpsrp`); all experiments in this paper use the Fed’s standard demo configuration (surplus-ratio targeting). Monetary policy can follow any of several published interest-rate rules selected by switch variables. The default in the Fed’s demonstration programs, and in this paper, is the *inertial Taylor rule* `rffintay`:

$$r_t^{ff} = \rho r_{t-1}^{ff} + (1 - \rho) \left[r_t^* + \bar{\pi}_t + 0.5 (\bar{\pi}_t - \pi_t^{targ}) + 1.0 \tilde{x}_t \right] + \varepsilon_t, \quad \rho = 0.85, \quad (6)$$

where $\bar{\pi}_t$ is the four-quarter moving average of core PCE inflation, π_t^{targ} the inflation target, r^* the policymakers’ perceived equilibrium real funds rate, $\tilde{x}_t$ the output gap (`xgap2`), and ε_t the additive shock term (`rffintay.aerr`) used to implement monetary-policy shocks. The coefficients (0.5 on the inflation gap, 1.0 on the output gap, smoothing 0.85) are as published in `model.xml`.

3 Implementation

The package (`frbus`) is roughly 700 lines of Python over `numpy`, `scipy`, `sympy` and `pandas`, organised as a pipeline from the Fed’s published artefacts to a compiled per-period solver. The public API mirrors the essentials of the Fed’s `pyfrbus`: `Frbus(path)`, `.init.trac(start, end, data)`, `.solve(start, end, data)` and `.exogenize([...])`, so that the Fed’s demo programs translate line-for-line.

3.1 Parsing the model and data

`parse.py` reads `model.xml` directly: variable names, endogenous/exogenous classification, per-equation coefficient values, and the VAR “standard” equation text in EViews syntax. `data.py` loads `LONGBASE.TXT` — the Fed’s quarterly database of history and staff projections, here the April 2026 vintage — into a `pandas` DataFrame on a quarterly `PeriodIndex`. `equations.py` and `lexing.py` normalise each equation: EViews constructs (`d(log(x),0,1)`, lag notation `x(-2)`, coefficient references `y_eq(i)`) are lexed, coefficients are substituted by value, the equation is flipped to residual form $F_i(\cdot) = 0$, and every equation is augmented with an additive tracking residual (add factor) `<endo>_trac` alongside the model’s own shock terms `<endo>_aerr`. Variable references are then index-substituted into array coordinates: current-period endogenous variables become entries of the unknown vector `x[i]`, and everything else (lags and exogenous variables) becomes `data[p, j]` references into the data matrix.

3.2 Symbolic differentiation and compilation

`symbolic.py` evaluates the substituted equation strings in a `sympy` namespace in which `x` and `data` are containers returning symbols literally named `x[i]` and `data[p, j]`; the string form of any derivative is then directly executable Python again. The analytic Jacobian is built by differentiating each residual F_i with respect to each unknown x_j appearing in it, stored as a sparse triplet list $(i, j, \partial F_i / \partial x_j)$, with the diagonal structurally present as in `pyfrbus`. Non-smooth primitives (`max`, `min`, an indicator function used by the policy-threshold equations) are given exact piecewise derivatives. `runtime.py` compiles the residual vector and Jacobian into ordinary Python functions evaluated with `numpy`; compilation happens once per data layout and is cached on the model object.

3.3 The per-period Newton solver

Because the VAR model is backward-looking across quarters, `solver.py` solves period by period. For each quarter the full 284-equation simultaneous system $F(x) = 0$ is solved by a damped Newton method: the Newton step solves the sparse linear system with an LU factorisation (`scipy.sparse.linalg.splu`) after diagonal row-scaling for conditioning, and a backtracking line search halves the step until the model evaluates cleanly (no domain errors or NaNs). Defaults are a step tolerance `xtol` = 10^{-8} and residual tolerance `rto1` = 5×10^{-4} ; the previous quarter’s solution is the initial guess. In practice a 20-quarter simulation solves in about three seconds on a laptop, including one-off symbolic compilation.

3.4 Add factors

`init_trac` implements FRB/US’s standard add-factoring (“tracking”) mechanism: for each quarter in the simulation window it evaluates every equation at the baseline data and stores the negated residual in the equation’s `_trac` variable, so that the baseline is an exact solution of the adjusted model. Shocks are then imposed on top of the add-factored baseline (e.g. via `_aerr` terms), and simulated paths are read as deviations from base. Section 4 shows this reproduction is exact to machine precision.

3.5 Differences from pyfrbus

The implementation follows `pyfrbus` semantics exactly but departs from it in engineering terms:

- **Modern dependencies.** No pin of `sympy` 1.3, no `scikit-umfpack/symengine` requirement, no pre-2.0 `pandas`; the package installs on current Python (3.10+) with wheels only. Sparse solves use `scipy`’s bundled SuperLU rather than UMFPACK.
- **Single-block Newton.** `pyfrbus` decomposes each period’s system into recursive and simultaneous blocks and solves them in sequence; this implementation solves the full simultaneous system in one Newton iteration loop. Both solve the same $F(x) = 0$, and Section 4 confirms the solutions agree to solver tolerance.
- **Scope.** MCE (rational-expectations) equation variants are not implemented — `Frbus(path, mce=...)` raises `NotImplementedError` — and the `mcontrol` trajectory-matching utility and stochastic simulations are likewise out of scope. The VAR model, add-factoring, exogenisation and shock simulation are complete.

4 Validation

The implementation is validated against the Fed’s own artefacts in two layers, both of which run as hard gates in the package’s test suite and continuous integration. All experiments use the VAR model over 2026Q1–2030Q4 with the Fed’s standard demo fiscal configuration (surplus-ratio targeting: `dfpdbl = 0`, `dfpsrp = 1`) and the April 2026 LONGBASE vintage.

4.1 Tracking invariant

After `init_trac`, solving the baseline must reproduce LONGBASE exactly for all 284 endogenous variables in all 20 quarters. This exercises the entire pipeline — parser, lexer, coefficient substitution, compiled residuals, Jacobian and Newton solver — against the Fed’s own data. Table 1 shows the result: the maximum absolute error across all 284×20 variable-quarters is 5.6×10^{-17} , i.e. machine precision.

Table 1: Tracking invariant: solved baseline vs. LONGBASE, 2026Q1–2030Q4

Metric	Value	Test gate
Max absolute error, all 284 endogenous $\times$ 20 quarters	5.6×10^{-17}	$< 10^{-8}$
Max relative error	5.6×10^{-17}	$< 10^{-10}$

4.2 Cross-validation against the Fed’s `pyfrbus`

The identical experiment — `init_trac` followed by a 100-basis-point shock to `rffintay_aerr` in 2026Q1, mirroring the Fed’s `demos/example1.py` — was run in both this implementation and the Fed’s `pyfrbus`, and all 284 endogenous variables were compared over all 20 quarters. The vendor reference was produced by `pyfrbus` running in a separate Python 3.11 environment with its Newton solver at `xtol = 10^{-8}`; a documented script regenerates it, recording the out-of-repo patches needed to run the vendor package on modern dependencies (an `splu` shim replacing `scikit-umfpack`, unpinning `sympy`, a fix to the vendor’s `Max/Min` derivative fallback under modern `symengine`, and `pandas < 2`).

Tables 2 and 3 summarise the agreement. The worst-agreeing variables are near-zero expectational gap terms (`zgap05`, relative difference 4.9×10^{-8} on values of order 10^{-4}), which reflect

Table 2: Cross-validation vs. pyfrbus: identical 100bp funds-rate shock, 2026Q1–2030Q4

Metric	Value	Test gate
Max absolute difference, all 284 endogenous $\times$ 20 quarters	6.0×10^{-9}	$< 10^{-6}$
Max relative difference (denominator clipped at 10^{-8})	4.9×10^{-8}	$< 10^{-6}$

Table 3: Cross-validation: agreement on key variables (max over 20 quarters)

Variable	Description	Max abs. diff.	Max rel. diff.
xgdp	Real GDP	1.5×10^{-10}	5.6×10^{-15}
pcpi	CPI price level	8.5×10^{-13}	2.5×10^{-15}
picxfe	Core PCE inflation	2.3×10^{-13}	1.2×10^{-13}
lur	Unemployment rate	4.0×10^{-13}	9.2×10^{-14}
rff	Federal funds rate	3.0×10^{-13}	1.1×10^{-13}

the two solvers’ respective step tolerances rather than any semantic difference; with the vendor solver at its default looser tolerance (`scipy.optimize.root`, `xtol` = 10^{-6}) the discrepancy is $\sim 1.7 \times 10^{-5}$ and again concentrated in the same terms. The headline variables agree to 10^{-10} or better in absolute terms. Since the vendor’s block-decomposed solver and this implementation’s single-block Newton solve the same $F(x) = 0$, agreement up to solver tolerance is the expected — and observed — outcome.

5 Simulation results and comparison with published FRB/US properties

5.1 A 100-basis-point monetary policy shock

The canonical FRB/US experiment adds 100 basis points to the additive shock term of the inertial Taylor rule (equation 6) in the first simulation quarter, on top of the add-factored baseline. Table 4 reports the full impulse responses from a fresh run of the released example (`examples/monetary_policy_shock.py`), as deviations from baseline over 2026Q1–2030Q4.

The funds rate rises by exactly 100 basis points on impact and then decays as the inertial rule unwinds the shock, undershooting mildly once activity and inflation have fallen. Real GDP declines with a hump shape to a trough of -0.55 per cent seven quarters after the shock (2027Q4), the unemployment rate peaks $+0.26$ percentage point above baseline in the same quarter, and core PCE inflation bottoms at -0.034 percentage point — the model’s well-known flat reduced-form Phillips curve at work.

5.2 Comparison with published FRB/US simulations

Brayton et al. (2014) report the effects of an unanticipated 1-percentage-point increase in the federal funds rate in FRB/US under both expectations options. Under VAR expectations — the relevant comparison — they show the output gap declining by a maximum of about 0.4 percentage point, inflation falling by up to about 0.08 percentage point, and activity remaining below baseline for five to six years, with the real funds rate elevated for two to three years;

Table 4: Impulse responses to a 100bp shock to the inertial Taylor rule (deviations from baseline)

Quarter	rff (pp)	xgdp (%)	lur (pp)	picxfe (pp)
2026Q1	1.000	0.001	−0.000	0.000
2026Q2	0.822	−0.187	0.099	−0.010
2026Q3	0.654	−0.290	0.151	−0.021
2026Q4	0.490	−0.436	0.210	−0.025
2027Q1	0.342	−0.483	0.227	−0.030
2027Q2	0.211	−0.528	0.246	−0.032
2027Q3	0.097	−0.544	0.254	−0.033
2027Q4	0.001	−0.551	0.258	−0.034
2028Q1	−0.079	−0.544	0.257	−0.033
2028Q2	−0.143	−0.526	0.250	−0.033
2028Q3	−0.191	−0.499	0.240	−0.032
2028Q4	−0.226	−0.464	0.226	−0.031
2029Q1	−0.248	−0.426	0.209	−0.030
2029Q2	−0.260	−0.384	0.190	−0.030
2029Q3	−0.262	−0.340	0.168	−0.029
2029Q4	−0.257	−0.296	0.146	−0.028
2030Q1	−0.245	−0.254	0.124	−0.027
2030Q2	−0.229	−0.214	0.101	−0.027
2030Q3	−0.209	−0.176	0.080	−0.026
2030Q4	−0.187	−0.142	0.059	−0.026

Bold entries mark the trough/peak responses. Source: fresh run of `examples/monetary_policy_shock.py` with the April 2026 LONGBASE vintage; identical to the cross-validated experiment of Table 2.

under MCE the same shock moves the output gap only about -0.1 percentage point. Our responses sit squarely in this range: a peak output loss of half a per cent about two years out, unemployment up a quarter point, and a modest inflation decline, with the economy still below baseline at the five-year horizon. Exact magnitudes are not expected to coincide: the published figures reflect an earlier model vintage and baseline (2014 versus the April 2026 `model.xml` and LONGBASE), a different design of the funds-rate perturbation, and a decade of re-estimation — the current vintage’s inflation response, in particular, is somewhat smaller than the 2014 figure, consistent with the further flattening of the estimated price block. The qualitative fingerprint of FRB/US under VAR expectations — hump-shaped and persistent real responses with small inflation movements — is exactly reproduced, and the quantitative agreement with the Fed’s own *code* on the identical vintage is at the 10^{-9} level (Section 4).

5.3 A government-purchases experiment

As a simple fiscal experiment, we exogenise real federal government purchases (`egfe`) and raise them by 1 per cent of baseline real GDP for eight quarters (2026Q1–2027Q4), leaving monetary policy on the inertial Taylor rule and fiscal closure on surplus-ratio targeting. Table 5 reports the results from a fresh run.

The impact multiplier is 0.74, averaging roughly 0.7 over the first year and eroding to 0.59 by the end of the second year as the inertial Taylor rule raises the funds rate by nearly 50 basis points and crowds out private demand; once the stimulus expires, output dips below baseline. This is consistent with the fiscal properties the Board has published for FRB/US: government-

Table 5: Government purchases +1% of GDP for 8 quarters: selected responses

Quarter	Shock (% of GDP)	xgdp (%)	lur (pp)	rff (pp)
2026Q1	1.0	0.74	−0.31	0.11
2026Q4	1.0	0.69	−0.31	0.34
2027Q4	1.0	0.59	−0.31	0.48
2028Q4	0.0	−0.30	0.10	0.12
2030Q4	0.0	−0.33	0.18	−0.21

Deviations from baseline. The implied spending multiplier is 0.74 on impact and averages about 0.72 over the first year, declining as the funds rate rises; output falls below baseline after the spending expires.

spending multipliers of roughly one when the funds rate is held fixed, and noticeably smaller — below one and declining with horizon — when monetary policy responds (Brayton et al., 2014). The experiment also illustrates the platform’s **exogenize** mechanism, which converts listed endogenous variables into exogenous instruments while keeping the remaining system simultaneous.

6 Limitations and conclusion

Four limitations should frame any use of this implementation.

VAR expectations only. Only the backward-looking VAR expectations version of FRB/US is implemented. The MCE (rational-expectations) equation variants — under which agents’ expectations are consistent with the model’s own future paths, and which the Fed uses for exercises such as forward-guidance and threshold-policy analysis — raise **NotImplementedError**. As Section 5 notes, the two expectations options can deliver materially different quantitative answers (an output-gap response of −0.4 versus −0.1 percentage point to the same funds-rate shock in Brayton et al., 2014), so results from this package speak only to the VAR configuration. The **mcontrol** trajectory-matching utility and stochastic simulations are likewise not implemented.

Data vintage. All results are tied to the April 2026 LONGBASE vintage and the contemporaneous `model.xml`. The Board re-estimates equations and revises the projection database over time; users should refresh the vendored artefacts from the Board’s website (Board of Governors of the Federal Reserve System, 2026a) and re-run the validation gates when doing so, since coefficients, baseline paths and hence impulse responses all shift with vintage.

Not an official product. FRB/US, its database and its documentation are produced and published by the Federal Reserve Board in the public domain; this reimplementaion is an independent open-source project of PolicyEngine and is neither produced, endorsed, nor reviewed by the Board. Fidelity claims rest entirely on the reproducible validation evidence of Section 4.

Model-inherent caveats. The implementation inherits FRB/US’s own modelling choices — estimated PAC dynamics, a flat reduced-form Phillips curve, policy closure by simple rules — and the usual caveats about using an estimated backward-looking system for counterfactuals far from the estimation sample apply with full force.

Within those bounds, the contribution is a small, dependable, continuously tested platform: the Fed’s public model definition and data, parsed and solved on a modern scientific-Python stack, reproducing the Fed’s own reference implementation to 10^{-9} on identical experiments and its published simulation properties in sign, shape and magnitude. Within PolicyEngine Macro it provides the U.S. macroeconomic backbone against which microsimulation-based tax and transfer analyses can be given dynamic, model-consistent macro context, and a template — artefact parsing, symbolic Jacobians, add-factoring, hard validation gates in CI — for bringing other public-sector macromodels into the open-source ecosystem.

References

- Board of Governors of the Federal Reserve System (2026a). FRB/US in Python. Web page. The pyfrbus simulation package, model.xml and the LONGBASE.TXT database (April 2026 vintage used here).
- Board of Governors of the Federal Reserve System (2026b). The FRB/US project. Web page. Model equations, data, documentation and simulation packages, published in the public domain.
- Brayton, F., Laubach, T., and Reifschneider, D. (2014). The FRB/US model: A tool for macroeconomic policy analysis. FEDS Notes, Board of Governors of the Federal Reserve System. <https://doi.org/10.17016/2380-7172.0012>.
- Brayton, F. and Tinsley, P. A. (1996). A guide to FRB/US: A macroeconomic model of the United States. Finance and Economics Discussion Series 1996-42, Board of Governors of the Federal Reserve System, Washington, D.C.
- Tinsley, P. A. (1993). Fitting both data and theories: Polynomial adjustment costs and error-correction decision rules. Finance and Economics Discussion Series 93-21, Board of Governors of the Federal Reserve System, Washington, D.C.
- Tinsley, P. A. (2002). Rational error correction. *Computational Economics*, 19(2):197–225.